Project Overview

This Directed Research Project started with a goal to build AI-resilient and autogradable Qiskit homework assignments for an undergraduate quantum software development course. The original motivation behind the project was that typical homework assignments and even projects can often be completed wholly by generative AI chatbots like ChatGPT or ClaudeAI when a student provides the assignment text or files to them.

The initial project focused on creating assignments where the output is dependent on actual execution of Qiskit code. These assignments would go on to implement techniques such as seeded generation, simulator outputs, noisy simulation, JSON autogradable submission files, circuit validation, and hidden reference-generation functions. The goal in mind was not to ban AI usage, but to build the homework assignments in a way that requires students to run, review, and discuss their unique results.

Over the course of the project, the focus shifted towards a clearer finding. It was observed through various experiments and tests that ordinary Qiskit assignments and even those with basic modifications for AI-deterrence remained vulnerable to completion by a minimally engaged ("Lazy") student wanting to simply complete the tasks using ChatGPT. This changed the emphasis from only making protections to keeping track of and documenting where ordinary and somewhat protected assignments failed, then making use of such for creation of better assignment architecture.

Main Highlight

Conventional Qiskit homework assignments, including assignments modified with AI-deterrence techniques, can for the most part be effectively completed by a lazy student using ChatGPT. The focus should rather be on requiring personalized execution, verification, interpretation and in-class checks of understanding instead of trying to prevent direct usage of AI tools.

The project's major result was found to be somewhat negative as many of the attempted techniques of homework protection did not reliably stop the "Lazy student + ChatGPT = completed assignment" scenario. However, this negative result is still useful as it guides towards methodological approaches that may be implemented in assignment structuring for improved defensibility.

Research Questions

RQ1: Which Qiskit homework tasks are most suitable for AI-resilient, auto-gradable assessment?

The experiments suggested that homework tasks are more suitable when they require personalized execution products rather than just written explanations. Stronger tasks include seeded circuits, customized measurement mappings, masked oracles, simulator and hardware comparisons, QPY circuit validation, and noisy result interpretation.

RQ2: Can seeded, execution-dependent Qiskit assignments be made reliable and accessible in Google Colab?

The homework assignments built suggest that this is indeed feasible for simulator-based (Qiskit) assignments. HW1-HW3 all followed a seed based structure where the student used their ID number to generate a personal configuration for their assignment and hidden reference functions can regenerate the correct instructor solution.

RQ3: Can an autograder verify circuit artifacts, expectation values, noisy-simulation summaries, and transpiler statistics with useful feedback?

This process was investigated in the current assignment packets by the creation of answers.json, JSON schemas, basic demo graders, etc. But more work remains for full processing and validation especially for noisy simulation grading and larger batch testing.

RQ4: How well do text-only LLM attempts perform when asked to solve these assignments without running Qiskit?

This research question became a central focus for the project. The experiments so far show that ChatGPT can often generate working or near working Qiskit code for most assignments. In most cases, the code/completed files required only minor setup fixes, such as installing missing libraries, or adjusting IBM runtime syntax due to outdated approaches the LLMs take in producing code. This means that text-only LLM assistance is strong enough to complete many regular assignments unless it requires very specific execution-dependent, personalized, or verification-based requirements and results that cannot be predicted.

RQ5: What design principles emerge for balancing accessibility, AI-resilience, grading reliability, and conceptual learning?

The project suggests that AI-resilient assignment design should not rely on hidden text, canary markers, vague anti-AI instructions, or even image embeddings. Instead, stronger designs should make use of seeded generation, machine-readable outputs, simulator execution, hardware/noise interpretation where appropriate, short reflection questions and in-person quizzes that connect numerical results to quantum concepts and a student's understanding.

Work Done

<u>Experimental tasks</u>

The experiments conducted included testing of ChatGPT's capabilities in aiding or completing Qiskit assignments such as:

- Quantum Phase estimation
- Quantum Fourier Transformation
- Deutsch-Jozsa
- Grover's Algorithm
- Simulator to IBM hardware conversions
- Bit ordering and measurements
- Canary-marker and hidden instruction or embeddings

A key finding included the ability for ChatGPT to produce working code for both Qiskit simulator and IBM hardware for lower-medium level quantum algorithms that would be expected for a course of this level. For example, as found in the QPE experiment, the simulator result produced the expected dominant bitstring and while the IBM hardware execution had a noisy result, the expected bitstring remained dominant. In the QFT experiment, the simulator produced the expected 101 output and the IBM hardware run on ibm kingston gave 101 as the dominant along with noise distributed among various other bitstrings.

Assignment prototypes (for technique development)
Three assignment packages were developed:

1. HW1: Seeded circuits, Bit flips, and Measurement Mapping
2. HW2: QFT / Inverse QFT hardware recovery
3. HW3: Seeded Deutsch Jozsa

The assignments formed the beginning of a tangible homework package that could be used to expand into more challenging or simply exploratory tasks. All assignments made use of key elements like seeded generation, guided tasks, and optional IBM hardware tasks to make it so that students can leave each and every assignment having learned more than they would have without this methodology.

Threat Model Used

The working threat model is not an expert malicious student. It is a minimally engaged ("Lazy") student who wants to complete the assignment quickly by pasting the prompt, notebook, or error messages into ChatGPT and copying the resulting code or explanation. The student may still run code cells and handle small setup issues, but they won't be partaking in learning the underlying concepts.

1. Paste (file, screenshot, text, etc) the homework into ChatGPT
2. Ask for completed Qiskit code or a filled notebook
3. Run the generated code in colab or jupyter
4. Ask ChatGPT to debug import or runtime errors
5. Copy generated explanations or reflection question answers
6. Submit outputs without an understanding of the quantum algorithm or task

Negative results summary

Attempted Protection/design	What happened	Project Takeaway
Standard Qiskit homework	ChatGPT often generated working or near-working code for textbook algorithms.	Generic algorithmic homework is highly vulnerable.
Anti-AI instructions	They clarified expectations but did not technically prevent AI-assisted completion.	Policy is useful but not sufficient.
Hidden text / invisible prompts	May reveal careless copying in some cases but can be removed or bypassed.	Detection trick, not a robust assignment design.
Canary markers	Markers can identify some AI-generated work but do not ensure understanding.	Limited practical value as a standalone protection.
Altered wording	ChatGPT still recognized the underlying quantum algorithm.	Surface-level changes do not change the core vulnerability.
Simulator-only execution	ChatGPT can write simulator code and often predict expected outputs.	Execution helps but needs personalization and validation.
IBM hardware execution	ChatGPT can write hardware code, but cannot know exact real counts without execution.	Useful evidence layer, but not full protection.
Generic reflection questions	ChatGPT can answer generic conceptual prompts well.	Reflections should be tied to the student's actual data.
Over-scaffolded notebooks	If solution functions are given, students do not need to reason through the task.	Keep answer-generation logic in instructor files.

Baseline Experiments With ChatGPT and Qiskit

A significant portion of the research centered on prompting ChatGPT to assist with or entirely generate Qiskit assignments. These trials proved valuable by illustrating the extent to which a student could simulate progress without first establishing a deep conceptual grasp of the underlying algorithms.

Quantum Phase Estimation

The initial proof-of-concept centered on Quantum Phase Estimation. ChatGPT was tasked with producing code for both local simulators and IBM hardware deployments. The resulting simulator code successfully yielded the anticipated dominant bitstring, with the final phase calculation aligning perfectly with theoretical expectations.

Furthermore, the IBM hardware implementation was successfully executed following standard authentication and environment configuration. While the hardware-generated histogram exhibited more noise than the simulator's output, the target bitstring remained the most frequent result. This experiment confirmed that ChatGPT is capable of facilitating the completion of course-level QPE tasks across both ideal and noisy environments.

Research significance: The QPE results indicate that mandating real hardware execution is not a definitive barrier to AI-driven assistance. Although students must still manage IBM account credentials and

navigate execution queues, the LLM can still generate the majority of the technical workflow and descriptive content.

Quantum Fourier Transform

The QFT investigation utilized an inverse-QFT recovery framework, as direct QFT produces superpositions that are less conducive to straightforward histogram comparison. This specific task involved preparing a known state, applying the QFT and its inverse, and then measuring the final output. In a 3-qubit QFT trial, the expected 101 output was returned for all 4096 simulator shots. A corresponding run on the ibm_kingston hardware produced 101 as the dominant bitstring with 3709 shots, while the remaining counts were scattered across other states due to inherent hardware noise.

Notable documentation points included the hardware retrieval cell stalling at the result call for several minutes before resolving—a practical challenge students are likely to encounter. Additionally, transpilation significantly increased the circuit depth, highlighting the structural divergence between abstract simulator circuits and their physical hardware counterparts.

Deutsch-Jozsa

Deutsch-Jozsa served as the foundation for HW3 due to its natural compatibility with seeded oracle layers. Rather than providing a uniform oracle, the assignment generates a unique mask based on a seed. This requires students to classify and build the oracle specifically for their case before testing the full circuit.

This custom masking approach increases resistance to generic AI prompts, as students must integrate their unique mask, construct the appropriate CNOT configuration, and correctly interpret the measured bitstring. Factors such as Qiskit's bit ordering and measurement mapping further complicate the task for a text-only assistant.

The development of HW3 included a student template, instructor solution, hidden reference functions, and a JSON-based grading schema. One documented accessibility issue was that the circuit drawing function required the pylatexenc package, which was noted as a setup requirement rather than a failure of the algorithm itself.

Grover's Algorithm

Grover's Algorithm was identified as a strong candidate for future homework due to the complexities introduced by custom marked states. While ChatGPT can generate simple Grover examples, it is more prone to errors when handling student-specific oracles, complex bit ordering, and the interpretation of noisy hardware results.

A proposed assignment for this algorithm would involve generating unique marked states, requiring students to build the corresponding oracle and diffuser, and running simulations alongside hardware tests. This design aims to identify the specific boundary where noise renders hardware results inconclusive, aligning with the goal of connecting execution to student understanding.

Hardware-Based Findings

Utilizing IBM quantum hardware provides a valuable, though imperfect, layer of deterrence against AI usage. While ChatGPT can script IBM Runtime workflows, it cannot predict specific hardware counts, transpiled circuit depths, or real-time backend conditions without actual execution.

The QPE and QFT trials demonstrated that small-scale circuits still yield clear dominant results despite noise, suggesting that hardware alone is not a "silver bullet." However, it does produce evidence that is much harder for a text-only LLM to accurately fabricate. Grading should therefore focus on artifacts like job IDs, raw counts, and the interpretation of success versus failure, rather than just exact numerical matches.

The project concludes that hardware should serve as a structured research extension. The simulator remains the core for reliable autograding, while hardware-based evidence reinforces AI-resilience and deeper conceptual learning.

Hardware Item	Importance
Backend name	Shows the student used a real or specified backend.
Job ID	Provides execution evidence that is difficult to invent convincingly.
Raw counts	Forces comparison against actual measured noise.
Dominant bitstring	Links hardware output to expected theoretical result.
Expected-result percentage	Supports a success/partial/inconclusive/failed classification.
Transpiled depth and gate counts	Shows backend-specific compilation effects.
Noise reflection	Connects raw data to quantum hardware limitations

Assignment Prototypes Built

HW1: Seeded Circuits, Bit Flips, and Measurement Mapping

This introductory assignment was crafted to establish the project's foundational infrastructure. It incorporates seeded personalization, bit-flip masking, and measurement mapping alongside simulator execution and JSON-based export functions. While the quantum algorithm itself is relatively simple, the primary educational objective is to familiarize students with the technical environment and the nuances of bit-ordering that are critical for subsequent tasks.

In this module, students are provided with a unique configuration based on their personal seed, which defines an initial bit pattern and specific mapping requirements. The task involves constructing a circuit with X gates, executing simulations, and calculating final values. Students must then interpret the resulting Qiskit count strings to generate a valid answers.json submission file.

The principal defense against AI-driven completion in this task is the complexity of bit ordering. Because Qiskit displays count strings in a reverse format (c[n-1]...c[0]), it often contradicts a student's or an LLM's natural expectation of q[0]...q[n-1]. By employing non-symmetric bitstrings and custom measurement maps, the assignment effectively highlights common errors made by both ChatGPT and students who lack a conceptual understanding of the framework.

HW2: QFT / Inverse QFT Hardware Recovery

The second assignment expands the established methodology to a sophisticated quantum algorithm. Students utilize a seeded input and measurement configuration to prepare states, apply the Quantum Fourier Transform followed by its inverse, and perform measurements. The simulator is expected to recover the original input bitstring as a validation of the process.

Deliverables for this task include simulator counts, dominant bitstrings, and technical metadata such as transpiled circuit depth and gate counts. An optional extension involves IBM hardware execution, where students must document backend specifics, Job IDs, and raw hardware counts to facilitate a deeper interpretation of physical hardware noise and execution success.

This prototype demonstrates that QFT tasks can be effectively autograded through inverse-recovery testing. However, it also underscores the risks of over-scaffolding; providing complete implementation logic within the student template can diminish the requirement for conceptual reasoning. Consequently, solution-generation functions should remain restricted to instructor-side files to maintain the assignment's academic integrity.

HW3: Seeded Deutsch-Jozsa

HW3 utilizes custom oracle masking as a robust layer of AI-deterrence. Students receive a seeded configuration and must first classify their specific oracle as constant or balanced before beginning construction. A nonzero mask identifies a balanced function, while an all-zero mask indicates a constant one, with an output offset providing an additional layer of complexity that does not affect the primary classification.

When constructing a balanced oracle, students must map each '1' in their mask to a corresponding CNOT gate directed toward the ancilla qubit. This personalized requirement ensures that a generic solution cannot be used for all students. After building the full circuit and executing simulations, students must interpret their unique bitstrings and export their results via the standardized JSON schema.

The development of this prototype confirmed the effectiveness of the workflow, though the student-facing template is designed to leave critical reasoning steps as "TODO" items. This requires students to actively classify oracles, build configurations, and provide written reflections, ensuring that the final submission reflects their own understanding rather than a copied prompt.

Hidden Reference Generation and Autograding

The implementation of hidden reference generation serves as a fundamental architectural pillar within the developed assignment prototypes. Rather than maintaining a static database of expected outcomes, the autograder dynamically utilizes student and assignment identifiers to recreate the specific seeded configuration. This enables the system to compute the instructor-side reference solution in real-time for direct comparison against the submitted JSON data.

This methodological approach effectively balances individual personalization with deterministic grading reliability. While every student interacts with a unique circuit or oracle, the instructor retains the ability to validate results with absolute precision. To maintain academic integrity, student notebooks are designed with guided instruction and TODO placeholders, while the core solution logic and validation routines are restricted to backend instructor files.

- generate_config(student_id): Instantiates the personalized version of the assignment.
- reference_answers(student_id): Calculates the anticipated solution from the instructor's perspective.
- validate_answers(student_answers, student_id): Audits the provided JSON against the reference artifacts.
- schema_hwX.json: Verifies the integrity of required fields and data types.
- grader_hwX_demo.py: Illustrates the practical application of the autograding framework.

LLM Boundaries and failure observations

Boundary / limitation	Description
No fixed qubit limit for code generation	ChatGPT can write code for many qubits, but this does not mean the circuit is practical.
Cannot know real hardware results without execution	It can predict theory but not actual backend noise, counts, or queue behavior.
Bit-ordering confusion	It may confuse q[0]...q[n-1] with displayed c[n-1]...c[0].
Custom oracle mistakes	It may write code that runs but marks the wrong state or uses the wrong mask.
Outdated Qiskit syntax	It may use old IBMQ/provider/runtime syntax that requires fixes.
Overconfident explanations	It may explain a circuit as correct while missing subtle issues measurement mapping.
Scaling and noise	As qubits and depth increase, hardware outputs may stop being clearly meaningful.
Generic reflections	It can easily answer broad conceptual questions unless they reference actual student data.

Emerging Principles for Assignment Design

- Avoid a reliance on simple policy-based prohibitions of AI; while they set expectations, they do not technically safeguard the assignment from automated completion.
- Implement seeded personalization to ensure that every student is presented with a distinct but reproducible configuration for their work.
- Adopt machine-readable submission formats, such as an answers.json file, to facilitate automated grading workflows.
- Restrict core solution-generation logic to instructor-side reference files to prevent students from accessing the underlying answer patterns.
- Utilize asymmetric bitstrings and customized measurement mappings to specifically reveal conceptual errors related to Qiskit's bit-ordering conventions.
- Mandate actual simulator-based execution products rather than permitting responses limited to textual or theoretical explanations.
- Incorporate IBM hardware execution as a structured layer of evidence rather than a standalone metric for grading.
- Structure reflection questions to depend directly on the student's unique execution artifacts, such as specific counts, transpiled depth, or backend data.
- Gather additional circuit artifacts, like QPY files, when the assessment requires more rigorous technical validation.
- Supplement digital submissions with brief in-class oral checks or quizzes to verify a student's genuine conceptual grasp.